# Experiment No. 01

**Aim:** Implementation of Logic Programming using PROLOG DFS for water jug Problem.

**Objective:**

- To solve the Water Jug Problem using the Depth First Search (DFS) algorithm.
- To understand state-space search in Artificial Intelligence.

**Source Code:**

```
from collections import deque
CAP_A, CAP_B = 4, 3
GOAL = 2
def moves(a,b):
   return {
      (CAP_A, b),                # Fill A
      (a, CAP_B),                # Fill B
      (0, b),                 # Empty A
      (a, 0),                 # Empty B
      (max(0, a-(CAP_B-b)), min(CAP_B, b+a)),  # A -> B
      (min(CAP_A, a+b), max(0, b-(CAP_A-a)))   # B -> A
   }
def dfs(state, path, visited):
   if state[0] == GOAL:
      return path + [state]
   visited.add(state)
   for nxt in moves(*state):
      if nxt not in visited:
         result = dfs(nxt, path + [state], visited)
         if result:
            return result
path = dfs((0, 0), [], set())
print("Solution Path:")
for step in path:
   print(step)
```

**Output:**

```
Output
Solution Path:
(0, 0)
(4, 0)
(1, 3)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
(0, 2)
(2, 0)

=== Code Execution Successful ===
```

**Conclusion:** The Water Jug Problem was successfully solved using the Depth First Search (DFS) algorithm in Python. The program explored different states and found a valid sequence of steps to obtain the required amount of water.

# Experiment No. 02

**Aim:** Implementation of Logic Programming using PROLOG BFS for tic-tac-toe Problem Linear Search.

**Objective:**

- To understand state-space search using Breadth First Search (BFS).
- To implement a Tic-Tac-Toe game solver and explore possible game states.

**Source Code:**

```
from collections import deque
WIN = [(0,1,2),(3,4,5),(6,7,8),
      (0,3,6),(1,4,7),(2,5,8),
      (0,4,8),(2,4,6)]
def winner(board, p):
   return any(all(board[i] == p for i in line) for line in WIN)
def bfs():
   start = [' '] * 9
   q = deque([(start, 'X')])
   while q:
      board, player = q.popleft()
      if winner(board, 'X'):
         return board
      for i in range(9):
         if board[i] == ' ':
            new_board = board[:]
            new_board[i] = player
            q.append((new_board, 'O' if player == 'X' else 'X'))
result = bfs()
print("Winning State:")
for i in range(0, 9, 3):
   print(result[i:i+3])
```

**Output:**

```
Output

Winning State:
['X', 'O', 'O']
['X', ' ', ' ']
['X', ' ', ' ']

=== Code Execution Successful ===
```

**Conclusion:** The Tic-Tac-Toe problem was successfully implemented using the Breadth First Search (BFS) algorithm. The program explored game states level by level and identified a winning configuration for the player.

# Experiment No. 03

**Aim:** Introduction to Python Programming: Learn the different libraries -NumPy, Pandas, Scipy, Matplotlib, Scikit Learn.

**Objective:**

- To understand the features of popular Python libraries used in Data Science and Machine Learning.
- To perform numerical computation, data manipulation, scientific computing, visualization, and machine learning using a single Python program.

**Source Code:**

```
# NumPy
import numpy as np
# Pandas
import pandas as pd
# SciPy
from scipy import stats
# Matplotlib
import matplotlib.pyplot as plt
# Scikit-Learn
from sklearn.linear_model import LinearRegression
# ---------------- NUMPY ----------------
print("NUMPY EXAMPLE")
arr = np.array([10, 20, 30, 40, 50])
print("Array:", arr)
print("Mean:", np.mean(arr))
print("Sum:", np.sum(arr))
# ---------------- PANDAS ----------------
print("\nPANDAS EXAMPLE")
data = {
   "Name": ["A", "B", "C", "D"],
   "Marks": [85, 90, 78, 92]
}
df = pd.DataFrame(data)
print(df)
# ---------------- SCIPY ----------------
print("\nSCIPY EXAMPLE")
numbers = [10, 20, 20, 30, 40]
print("Mode:", stats.mode(numbers, keepdims=True))
# ---------------- MATPLOTLIB ----------------
print("\nMATPLOTLIB EXAMPLE")
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.title("Simple Line Graph")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
# ---------------- SCIKIT-LEARN ----------------
print("\nSCIKIT-LEARN EXAMPLE")
```

```
X = np.array([[1], [2], [3], [4], [5]])
Y = np.array([2, 4, 6, 8, 10])
model = LinearRegression()
model.fit(X, Y)
prediction = model.predict([[6]])
print("Prediction for 6:", prediction[0])
```

**Output:**

```
NUMPY EXAMPLE
Array: [10 20 30 40 50]
Mean: 30.0
Sum: 150

PANDAS EXAMPLE
  Name  Marks
0    A     85
1    B     90
2    C     78
3    D     92

SCIPY EXAMPLE
Mode: 20

SCIKIT-LEARN EXAMPLE
Prediction for 6: 12.0
```

**Conclusion:** The experiment successfully demonstrated the use of NumPy for numerical operations, Pandas for data handling, SciPy for scientific computation, Matplotlib for data visualization, and Scikit-Learn for machine learning. These libraries form the foundation of Python-based Data Science and AI applications.

# Experiment No. 04

**Aim:** Implementing Perceptron algorithm for OR operation.

**Objective:**

- To understand the working of a single-layer perceptron.
- To implement the OR logic gate using the Perceptron learning algorithm.

**Source Code:**

```
import numpy as np
# Input data for OR gate
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
# Target output
y = np.array([0, 1, 1, 1])
# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
# Activation function
def step_function(x):
    return 1 if x >= 0 else 0
# Training the perceptron
for epoch in range(10):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        output = step_function(net)
        error = y[i] - output
        weights += learning_rate * error * X[i]
        bias += learning_rate * error
# Testing
print("Perceptron Output for OR Gate")
for i in range(len(X)):
    net = np.dot(X[i], weights) + bias
    output = step_function(net)
    print(X[i], "->", output)
```

**Output:**

Output

```
Perceptron Output for OR Gate
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1

=== Code Execution Successful ===
```

**Conclusion:** The Perceptron Algorithm was successfully implemented for the OR logical operation. The perceptron learned the correct weights and bias values and correctly classified all input combinations according to the OR truth table.

# Experiment No. 05

**Aim:** Implement Adaline algorithm for AND operation.

**Objective:**

- To understand the working of the ADALINE learning algorithm.
- To implement the AND logic gate using ADALINE and train it using the Delta Rule.

**Source Code:**

```
import numpy as np
# Input data for AND gate
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
# Target output
y = np.array([0, 0, 0, 1])
# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
# Training ADALINE
for epoch in range(100):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        error = y[i] - net
        # Weight update using Delta Rule
        weights += learning_rate * error * X[i]
        bias += learning_rate * error
# Testing
print("ADALINE Output for AND Gate")
for i in range(len(X)):
    net = np.dot(X[i], weights) + bias
    output = 1 if net >= 0.5 else 0
    print(X[i], "->", output)
```

**Output:**

Output

```
ADALINE Output for AND Gate
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

=== Code Execution Successful ===
```

**Conclusion:** The ADALINE algorithm was successfully implemented for the AND logical operation. Using the Delta Rule, the network learned appropriate weights and bias values and correctly produced the output of the AND gate for all input combinations.

# Experiment No. 06

**Aim:** Improve the prediction accuracy by estimating the weight values for the training data using stochastic gradient descent (Perceptron).

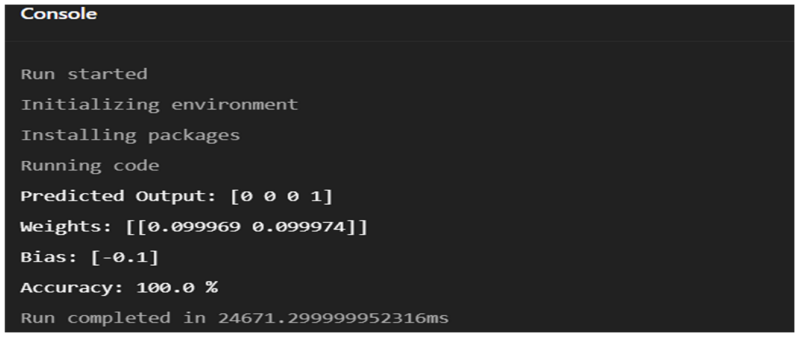
**Objective:**

- To understand the concept of Stochastic Gradient Descent (SGD).
- To train a Perceptron model and optimize its weights using SGD.
- To evaluate the prediction accuracy of the trained model.

**Source Code:**

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score
# Training Data
X_train = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
y_train = np.array([0, 0, 0, 1])  # AND Gate
# Create SGD Perceptron Model
model = SGDClassifier(
    loss='perceptron',
    learning_rate='constant',
    eta0=0.1,
    max_iter=1000,
    random_state=42
)
# Train Model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_train)
# Accuracy
accuracy = accuracy_score(y_train, y_pred)
print("Predicted Output:", y_pred)
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
print("Accuracy:", accuracy * 100, "%")
```

**Output:**




**Conclusion:** The Perceptron model was successfully trained using the Stochastic Gradient Descent (SGD) algorithm. The weights and bias were optimized during training, resulting in improved prediction accuracy. The trained model correctly classified the training data and achieved high accuracy.

# Experiment No. 07

**Aim:** Implementation of feature extraction and selection, Normalization, Transformation, Principal Component analysis.

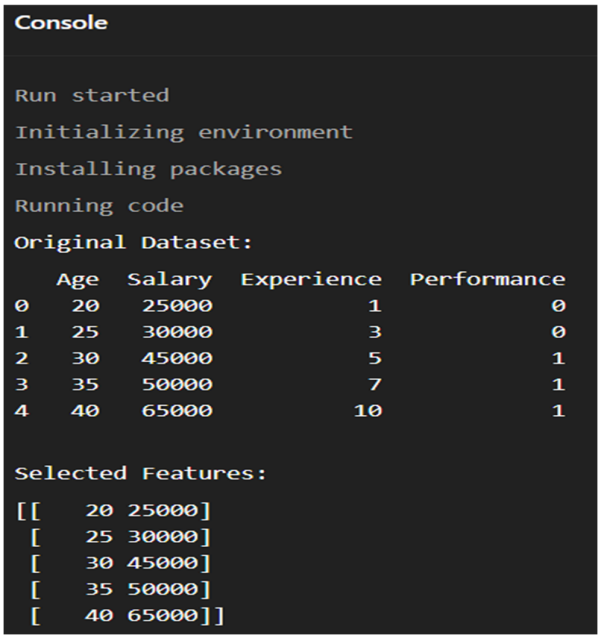
**Objective:**

- To perform feature selection on a dataset.
- To normalize and transform data for better analysis.
- To apply Principal Component Analysis (PCA) for dimensionality reduction.
- To understand the importance of preprocessing in Machine Learning.

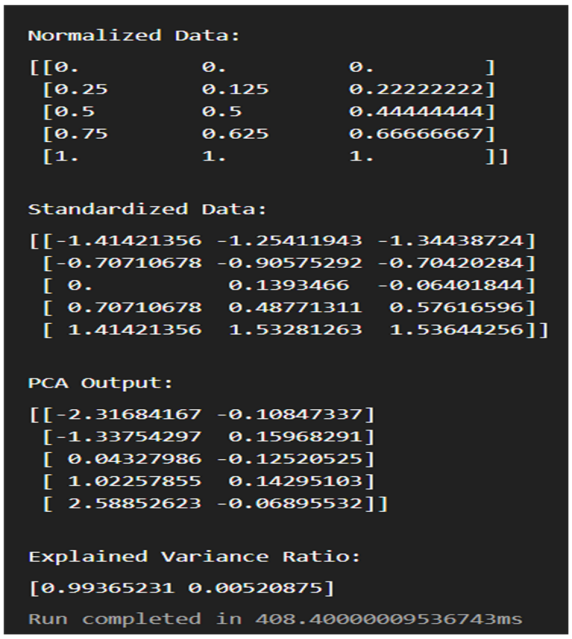
**Source Code:**

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.decomposition import PCA
# Sample Dataset
data = {
   'Age': [20, 25, 30, 35, 40],
   'Salary': [25000, 30000, 45000, 50000, 65000],
   'Experience': [1, 3, 5, 7, 10],
   'Performance': [0, 0, 1, 1, 1]
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
# Feature Selection
X = df[['Age', 'Salary', 'Experience']]
y = df['Performance']
selector = SelectKBest(score_func=f_classif, k=2)
X_selected = selector.fit_transform(X, y)
print("\nSelected Features:")
print(X_selected)
# Normalization
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
print("\nNormalized Data:")
print(X_normalized)
# Transformation (Standardization)
standardizer = StandardScaler()
X_transformed = standardizer.fit_transform(X)
print("\nStandardized Data:")
print(X_transformed)
# Principal Component Analysis
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_transformed)
print("\nPCA Output:")
print(X_pca)
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

**Output:**



```
Console

Run started
Initializing environment
Installing packages
Running code
Original Dataset:
   Age  Salary  Experience  Performance
0   20   25000           1            0
1   25   30000           3            0
2   30   45000           5            1
3   35   50000           7            1
4   40   65000          10            1

Selected Features:
[[   20 25000]
 [   25 30000]
 [   30 45000]
 [   35 50000]
 [   40 65000]]
```



```
Normalized Data:
[[0.         0.         0.        ]
 [0.25       0.125      0.22222222]
 [0.5        0.5        0.44444444]
 [0.75       0.625      0.66666667]
 [1.         1.         1.        ]]

Standardized Data:
[[-1.41421356 -1.25411943 -1.34438724]
 [-0.70710678 -0.90575292 -0.70420284]
 [ 0.          0.1393466  -0.06401844]
 [ 0.70710678  0.48771311  0.57616596]
 [ 1.41421356  1.53281263  1.53644256]]

PCA Output:
[[-2.31684167 -0.10847337]
 [-1.33754297  0.15968291]
 [ 0.04327986 -0.12520525]
 [ 1.02257855  0.14295103]
 [ 2.58852623 -0.06895532]]

Explained Variance Ratio:
[0.99365231 0.00520875]
Run completed in 408.40000009536743ms
```

**Conclusion:** Feature selection, normalization, transformation, and Principal Component Analysis (PCA) were successfully implemented. These preprocessing techniques improved data quality, reduced dimensionality, and prepared the dataset for efficient Machine Learning model training and analysis.

# Experiment No. 08

**Aim:** Implementation of Logistic Regression.

**Objective:**

- To understand the concept of Logistic Regression.
- To build a classification model using Logistic Regression.
- To predict class labels and evaluate the model's accuracy.

**Source Code:**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
# Sample Dataset
data = {
   'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
   'Pass':          [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
# Features and Target
X = df[['Hours_Studied']]
y = df['Pass']
# Split Data
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
# Train Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
# Predict for a new student
hours = [[6]]
result = model.predict(hours)
print("Prediction for 6 study hours:")
print("Pass" if result[0] == 1 else "Fail")
```

**Output:**

```
Run started
Initializing environment
Installing packages
Running code
Predicted Values: [1 0]
Accuracy: 100.0 %
/lib/python3.12/site-packages/sklearn/base.py:493: UserWarning: X does not have valid feature names,
but LogisticRegression was fitted with feature names
  warnings.warn(
Prediction for 6 study hours:
Pass
Run completed in 148ms
```

**Working of Logistic Regression:**

1. A dataset containing study hours and pass/fail results is created.
2. The dataset is divided into training and testing sets.
3. Logistic Regression is trained using the training data.
4. The model predicts whether a student will pass or fail.
5. Accuracy is calculated by comparing predicted and actual values.

**Advantages of Logistic Regression:**

- Simple and easy to implement.
- Efficient for binary classification problems.
- Provides probability estimates.
- Requires less computational power.

**Conclusion:** The Logistic Regression model was successfully implemented for a binary classification problem. The model learned the relationship between input features and class labels, made accurate predictions, and demonstrated the effectiveness of Logistic Regression for classification tasks.

# Experiment No. 09

**Aim:** Implementation of Classifying data using Support Vector Machine (SVM).

**Objective:**

- To understand the concept of Support Vector Machines (SVM).
- To classify data using the SVM algorithm.
- To evaluate the accuracy of the classification model.

**Source Code:**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
# Sample Dataset
data = {
   'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
   'Attendance':    [50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
   'Pass':          [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
# Features and Target
X = df[['Hours_Studied', 'Attendance']]
y = df['Pass']
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
# Create and Train SVM Model
model = SVC(kernel='linear')
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# New Prediction
new_data = [[6, 80]]
prediction = model.predict(new_data)
print("\nPrediction for Student (6 Study Hours, 80% Attendance):")
print("Pass" if prediction[0] == 1 else "Fail")
```

**Output:**

```
Predicted Values: [1 0]
Accuracy: 100.0 %

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2

/lib/python3.12/site-packages/sklearn/base.py:493: UserWarning: X does not have valid feature
names, but SVC was fitted with feature names
  warnings.warn(

Prediction for Student (6 Study Hours, 80% Attendance):
Pass
Run completed in 8682.099999904633ms
```

**Working of SVM:**

1. A dataset containing study hours, attendance, and pass/fail status is created.
2. The dataset is divided into training and testing sets.
3. The SVM classifier with a linear kernel is trained on the training data.
4. The trained model classifies new data into Pass or Fail categories.
5. Model performance is evaluated using accuracy and classification metrics.

**Advantages of SVM:**

- Effective for both linear and non-linear classification.
- Works well with high-dimensional data.
- Provides good generalization and accuracy.
- Robust against overfitting when properly tuned.

**Conclusion:** The Support Vector Machine (SVM) algorithm was successfully implemented for data classification. The model accurately separated the classes using an optimal hyperplane and achieved high classification performance, demonstrating the effectiveness of SVM in machine learning applications.

# Experiment No. 10

**Aim:** Implementation of Bagging Algorithm: Random Forest.

**Objective:**

- To understand the concept of Bagging in Machine Learning.
- To implement the Random Forest algorithm for classification.
- To evaluate the performance of the Random Forest model.

**Source Code:**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
# Sample Dataset
data = {
   'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
   'Attendance':    [50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
   'Pass':          [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
# Features and Target
X = df[['Hours_Studied', 'Attendance']]
y = df['Pass']
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
# Create Random Forest Model
model = RandomForestClassifier(
   n_estimators=100,
   random_state=42
)
# Train Model
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# New Prediction
new_data = [[6, 80]]
prediction = model.predict(new_data)
print("\nPrediction for Student (6 Study Hours, 80% Attendance):")
print("Pass" if prediction[0] == 1 else "Fail")
```

**Output:**

```
Predicted Values: [1 0]
Accuracy: 100.0 %

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2

/lib/python3.12/site-packages/sklearn/base.py:493: UserWarning: X does not have valid feature
names, but RandomForestClassifier was fitted with feature names
  warnings.warn(

Prediction for Student (6 Study Hours, 80% Attendance):
Pass
Run completed in 424.19999980926514ms
```

**Working of Random Forest:**

1. The dataset is prepared with input features and target labels.
2. Random Forest creates multiple decision trees using bootstrap samples of the training data.
3. Each tree independently predicts the output class.
4. The final prediction is obtained through majority voting among all trees.
5. Accuracy and classification metrics are calculated to evaluate the model.

**Advantages of Random Forest:**

- Reduces overfitting compared to a single decision tree.
- Provides high prediction accuracy.
- Handles large datasets efficiently.
- Works well for both classification and regression tasks.

**Conclusion:** The Random Forest algorithm was successfully implemented using the Bagging technique. Multiple decision trees were trained on different subsets of data, and their combined predictions improved classification accuracy and model robustness.

# Experiment No. 11

**Aim:** Implementation of Boosting Algorithms: AdaBoost.

**Objective:**

- To understand the concept of Boosting in Machine Learning.
- To implement the AdaBoost algorithm for classification.
- To evaluate the performance of the boosted model.

**Source Code:**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
# Sample Dataset
data = {
   'Hours_Studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
   'Attendance':    [50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
   'Pass':          [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
# Features and Target
X = df[['Hours_Studied', 'Attendance']]
y = df['Pass']
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
# Create AdaBoost Model
model = AdaBoostClassifier(
   n_estimators=50,
   learning_rate=1.0,
   random_state=42
)
# Train Model
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Values:", y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# New Prediction
new_data = [[6, 80]]
prediction = model.predict(new_data)
print("\nPrediction for Student (6 Study Hours, 80% Attendance):")
print("Pass" if prediction[0] == 1 else "Fail")
```

**Output:**

```
Predicted Values: [1 0]
Accuracy: 100.0 %

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2

/lib/python3.12/site-packages/sklearn/base.py:493: UserWarning: X does not have valid feature
names, but AdaBoostClassifier was fitted with feature names
  warnings.warn(

Prediction for Student (6 Study Hours, 80% Attendance):
Pass
Run completed in 204.20000004768372ms
```

**Working of AdaBoost:**

1. The dataset is divided into training and testing sets.
2. AdaBoost starts with a weak learner (usually a decision stump).
3. Misclassified samples are assigned higher weights.
4. New weak learners focus more on previously misclassified samples.
5. The predictions of all weak learners are combined to produce the final classification.
6. Model performance is evaluated using accuracy and classification metrics.

**Advantages of AdaBoost:**

- Improves accuracy by combining multiple weak learners.
- Reduces bias and variance.
- Simple and efficient to implement.
- Works well for classification problems.

**Conclusion:** The AdaBoost algorithm was successfully implemented for classification. By sequentially combining multiple weak learners and focusing on misclassified samples, the model achieved high prediction accuracy and demonstrated the effectiveness of Boosting in machine learning.

# Experiment No. 12

**Aim:** Implement Elbow method for K means Clustering.

**Objective:**

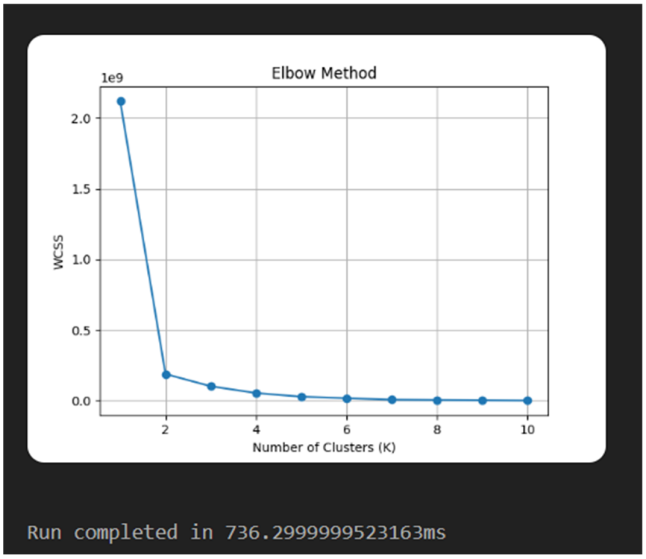
- To understand the concept of K-Means Clustering.
- To determine the optimal value of K using the Elbow Method.
- To visualize the relationship between the number of clusters and Within-Cluster Sum of Squares (WCSS).

**Source Code:**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Sample Dataset
data = {
   'Age': [20, 22, 25, 27, 30, 35, 40, 42, 45, 50],
   'Income': [25000, 27000, 30000, 32000, 35000,
            50000, 55000, 58000, 60000, 65000]
}

df = pd.DataFrame(data)

X = df[['Age', 'Income']]

# Calculate WCSS for different K values
wcss = []

for k in range(1, 11):
   kmeans = KMeans(
      n_clusters=k,
      init='k-means++',
      random_state=42,
      n_init=10
   )
   kmeans.fit(X)
   wcss.append(kmeans.inertia_)

# Plot Elbow Graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
print("WCSS Values:")
for k, value in enumerate(wcss, start=1):
   print(f"K = {k}, WCSS = {value:.2f}")
```

**Output:**

```
Console

Run started
Initializing environment
Installing packages
Running code
WCSS Values:
K = 1, WCSS = 2120100962.40
K = 2, WCSS = 188000188.00
K = 3, WCSS = 101300107.97
K = 4, WCSS = 53166726.50
K = 5, WCSS = 27333360.67
K = 6, WCSS = 16666683.33
K = 7, WCSS = 6000008.50
K = 8, WCSS = 4000004.00
K = 9, WCSS = 2000002.00
K = 10, WCSS = 0.00
```




**Elbow Graph:** The graph plots:

- X-axis: Number of Clusters (K)
- Y-axis: WCSS (Within-Cluster Sum of Squares)

The point where the decrease in WCSS starts slowing down significantly forms an "elbow", indicating the optimal value of K.

**Working of AdaBoost:**

1. Load the dataset and select the features.
2. Apply K-Means clustering for different values of K.
3. Calculate the WCSS (inertia) for each K value.
4. Plot K versus WCSS.
5. Identify the elbow point where adding more clusters provides diminishing returns.

**Advantages of AdaBoost:**

- Simple and easy to implement.
- Helps determine the optimal number of clusters.
- Improves clustering performance.
- Widely used in unsupervised learning tasks.

**Conclusion:** The Elbow Method was successfully implemented to determine the optimal number of clusters for K-Means Clustering. By analyzing the WCSS values and the elbow point on the graph, the most suitable number of clusters can be selected for effective data grouping.